

Comparative Benchmarking of TensorFlow Serving vs. TorchServe on Azure

Sai Vishnu
Senagasetty
– 2284855

Kausthubh
Reddy Eleti –
2297857

Nitish
Dadapuram -
2304669

Problem Statement & Objectives

- ▶ How do TensorFlow Serving and TorchServe compare in real-world deployment settings when serving the same models under identical conditions ?
- ▶ Which framework provides better scalability and efficiency for inference workloads, and under what conditions ?

Objectives

- ▶ Train and export the same CNN and RNN models in both TensorFlow and PyTorch, and deploy them using TensorFlow Serving and TorchServe respectively
- ▶ Use Azure Machine Learning managed online endpoints to host each model, ensuring the deployment pipeline is consistent for both frameworks.
- ▶ Generate realistic API request loads (light, medium, heavy) using stochastic request arrival patterns to mimic real user traffic.
- ▶ Collect and compare average latency per request, throughput (requests per second served), and success vs. failure rates for each framework under each load level.

TensorFlow vs PyTorch – Intro

Feature	TensorFlow	PyTorch
Origin	Developed by Google	Developed by Facebook (Meta)
Programming Style	Static graph (declarative) – good for deployment	Dynamic graph (eager execution) – great for research
Serving Tool	TensorFlow Serving (C++ backend, highly optimized)	TorchServe (Python-based, customizable)
Ease of Debugging	Slightly harder due to static graph model	Easier and more intuitive
Integration in Azure	Supported via curated environments	Requires custom environments

Dataset Overview

Fashion MNIST (Image Classification)

- A benchmark dataset for image classification introduced by Zalando Research.
- **Total Samples:** 70,000 grayscale images (60,000 train, 10,000 test)
- **Image Size:** 28×28 pixels (single channel)
- **Classes:** 10 clothing categories
 - T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot
- **Why?**
 - Small and efficient for benchmarking.
 - Ideal to evaluate serving latency for lightweight CNN models.

IMDB Movie Reviews (Text Sentiment Classification)

- A binary sentiment classification dataset from the Internet Movie Database.
- **Total Samples:** 50,000 movie reviews (25k positive, 25k negative)
- **Preprocessed Format:** Each review is represented as a sequence of word indices (integers).
- **Why?**
 - Representative of real-world NLP workloads.
 - Useful for testing how well serving systems handle sequence-based models (e.g., RNNs).

Model Architectures & Export Formats

CNN Image Classifier – Fashion-MNIST (Computer Vision)

- Architecture: LeNet-5–style CNN (Convolution + Dense layers)
- Task: Classify clothing images into categories
- Frameworks Used: TensorFlow and PyTorch
- Export Formats:
 - TensorFlow: SavedModel format (.pb graph + variables directory)
 - PyTorch: TorchScript format (.pt file)

RNN Sentiment Classifier – IMDB (Natural Language Processing)

- Architecture: Embedding + LSTM + Dense
- Task: Binary sentiment classification of movie reviews
- Frameworks Used: TensorFlow (via Keras) and PyTorch (nn.LSTM)
- Export Formats:
 - TensorFlow: SavedModel format
 - PyTorch: TorchScript format (.pt file)

Deployment Pipeline on Azure ML

1. Model Registration
2. Environment & Dependencies
3. Managed Online Endpoints
4. Inference & Scoring Scripts
5. Access, Traffic, and Compute

Deployment Pipeline on Azure ML

1. Model Registration

- **Trained Models:** CNN (image) and RNN (text) models using TensorFlow and PyTorch.
- **Registration:** Uploaded/exported as versioned artifacts:
- TensorFlow: **SavedModel** format
- PyTorch: **TorchScript** (.pt file)
- **Why important?** Ensures traceability and reproducibility.

Why Proper Registration & Versioning Matter:

- Traceability
- Safe Deployment

```
from azure.ai.ml.entities import Model
model = Model(
    path="path/to/model_dir_or_file",
    name="my_model_name",
    version="1", # or let Azure auto-increment
    description="Model description"
)
ml_client.models.create_or_update(model)
```


Deployment Pipeline on Azure ML

2. Environment & Dependencies

- **TensorFlow Serving:** Used Azure ML's curated Docker environment (tensorflow-2.16-cuda12).
- **TorchServe:** Created a custom environment (Conda YAML) with PyTorch, TorchServe, and required packages.
- **Key point:** Matching environments prevent compatibility issues during inference.

Why is this step important?

- Avoiding Unnecessary Dependencies
- Ensuring Compatibility
- “out of disk” errors.

```
from azure.ai.ml.entities import ManagedOnlineDeployment

deployment = ManagedOnlineDeployment(
    name="blue",
    model="tf_rnn_sentiment_saved_model:2",
    environment="azureml://registries/azureml/environments/tensorflow-2.16-cuda12/versions/6",
    code_configuration={"code": "./", "scoring_script": "score_tf_rnn.py"},
    instance_type="Standard_F2s_v2"
)
ml_client.online_deployments.begin_create_or_update(deployment)
```

```
from azure.ai.ml.entities import Environment

env = Environment(
    name="pytorch-env",
    version="1",
    conda_file="./conda.yml", # Path to the YAML above
    image="mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04"
)
ml_client.environments.create_or_update(env)
```


Deployment Pipeline on Azure ML

3. Managed Online Endpoints

- **Endpoints Created:**
- CNN-TensorFlow
- CNN-TorchServe
- RNN-TensorFlow
- RNN-TorchServe
- **How it works:** Each endpoint is an isolated, containerized deployment accessible via REST API.

```
from azure.ai.ml.entities import ManagedOnlineDeployment

deployment = ManagedOnlineDeployment(
    name="blue", # Use a color or custom name
    endpoint_name="cnn-tf-endpoint",
    model="tf_cnn_model:1", # Model name:version
    environment="tf-serving-env", # Or "torchserve-env"
    code_configuration={
        "code": "./src", # Folder with your score_tf.py or equivalent
        "scoring_script": "score_tf.py" # or score_pytorch.py etc.
    },
    instance_type="Standard_F2s_v2",
    instance_count=1
)

ml_client.online_deployments.begin_create_or_update(deployment).result()
```

Deployment Pipeline on Azure ML

4. Inference & Scoring Scripts

- **Entry Scripts:**
- TensorFlow:
score_tf.py,
score_tf_rnn.py
- PyTorch:
score_pytorch_cnn.py,
score_rnn_pytorch.py
- **Responsibilities:**
Load the model,
preprocess input, run
inference, return
predictions in JSON.

```
from azure.ai.ml.entities import ManagedOnlineDeployment

deployment = ManagedOnlineDeployment(
    name="blue", # Use a color or custom name
    endpoint_name="cnn-tf-endpoint",
    model="tf_cnn_model:1", # Model name:version
    environment="tf-serving-env", # Or "torchserve-env"
    code_configuration={
        "code": "./src", # Folder with your score_tf.py or equivalent
        "scoring_script": "score_tf.py" # or score_pytorch.py etc.
    },
    instance_type="Standard_F2s_v2",
    instance_count=1
)
ml_client.online_deployments.begin_create_or_update(deployment).result()
```

Deployment Pipeline on Azure ML

5. Access, Traffic, and Compute

- **REST API Access:** HTTP POST for real-time predictions (images for CNN, text for RNN).
- **Traffic Routing:** 100% traffic routed to the primary deployment (no canary/blue-green in this study).
- **Compute:** Standard_F2s_v2 VM (2 vCPU, 4GB RAM) for all endpoints—optimized for cost and quota.

```
import requests
import numpy as np

# Replace with your endpoint and key
endpoint = "https://<your-endpoint>.azurewebsites.net/score"
api_key = "<your-api-key>"

# Create a fake 28x28 image sample (normalized float32)
sample_input = np.random.rand(1, 28, 28).astype(np.float32).tolist()

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}

response = requests.post(
    url=endpoint,
    headers=headers,
    json={"data": sample_input}
)

print("CNN Prediction:", response.json())
```

```
np.random.randint(1, 10000, size=(1, 200)).astype(np.float32).tolist()
```


Load Testing Methodology (Overview)

1. Request Generation

- Developed a multithreaded Python load generator using *"requests"* library and *"threading"* module
- Each thread:
 - Sends continuous HTTP POST requests to a selected model endpoint
 - Randomly selects input data (image or text)
 - Records round-trip response time and status

2. Stochastic Traffic Simulation

- Used fixed sleep intervals to approximate request rates
- Achieved target RPS (requests per second) by adjusting:
 - Number of threads
 - Sleep duration between requests
- Note: Uniform timing used (Poisson process optional for future)

3. Load Levels (Scalability Testing)

- Light Load: ~5 RPS for 60 seconds
- Medium Load: ~20 RPS for 60 seconds
- Heavy Load: ~50 RPS for 30 seconds

Load Testing Methodology (Execution & Logging)

4. Logging and Metrics Collection

- For each request:
 - Logged: Timestamp, latency, HTTP status (success/failure)
- Output: CSV log files per configuration
 - Example: cnn_tf_light.csv, rnn_torch_heavy.csv
- Post-processing:
 - Aggregated logs to compute average latency total throughput

5. Infrastructure Considerations

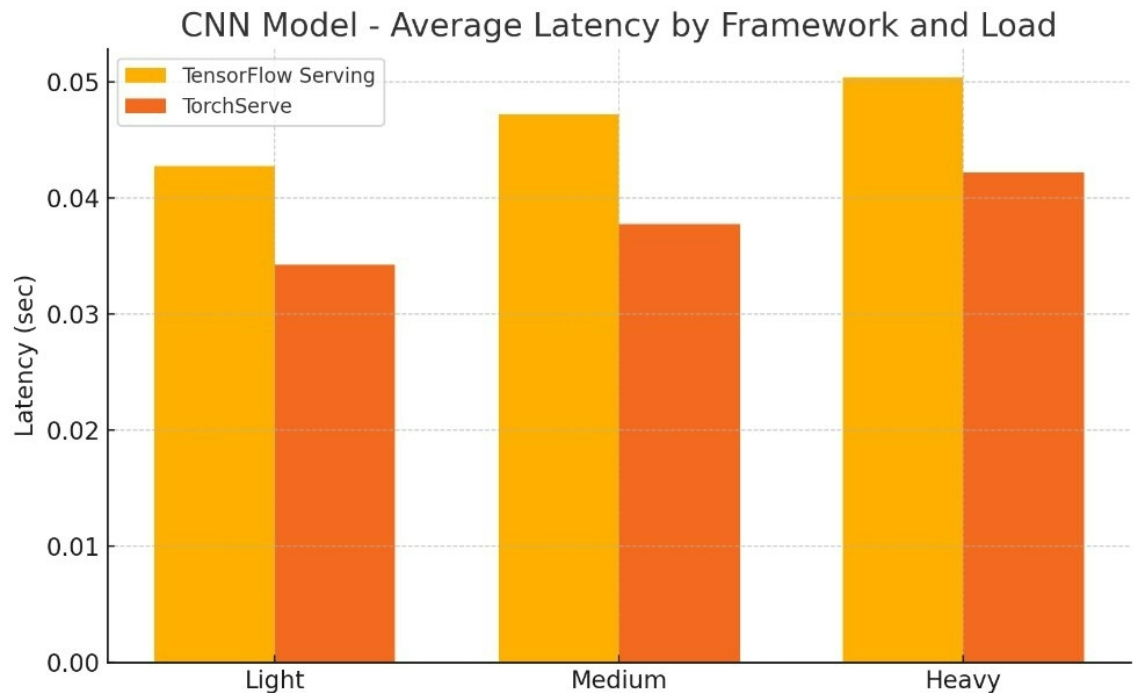
- Load generator executed from a client VM in the same Azure region as the model endpoints
- Isolated test runs:
 - Only one endpoint tested at a time
 - Avoided cross-traffic interference
- Minimal noise from client-side garbage collection or overhead

Performance Results Overview

- ▶ Compared TensorFlow Serving and TorchServe on CNN (image) and RNN (text) models
- ▶ Evaluated under 3 load levels: Light (5 RPS), Medium (20 RPS), Heavy (50 RPS)
- ▶ **Key Metrics:**
 - ▶ Average Latency per request (ms)
 - ▶ Throughput (requests per second)
 - ▶ 100% success rate across all scenarios — no request failures
 - ▶ Goal: Assess latency, scalability, and reliability for deployment

CNN Model – Latency Comparison

- ▶ **Model: Fashion-MNIST CNN Image Classifier**
- ▶ **TorchServe showed lower average latency across all load levels:**
- ▶ **Light Load: 34.3 ms (Torch) vs 42.8 ms (TF)**
- ▶ **Medium Load: 37.8 ms (Torch) vs 47.2 ms (TF)**
- ▶ **Heavy Load: 42.2 ms (Torch) vs 50.4 ms (TF)**
- ▶ **Interpretation:**
- ▶ **TorchServe's TorchScript deployment had less overhead**
- ▶ **Suitable for vision tasks in PyTorch ecosystem**

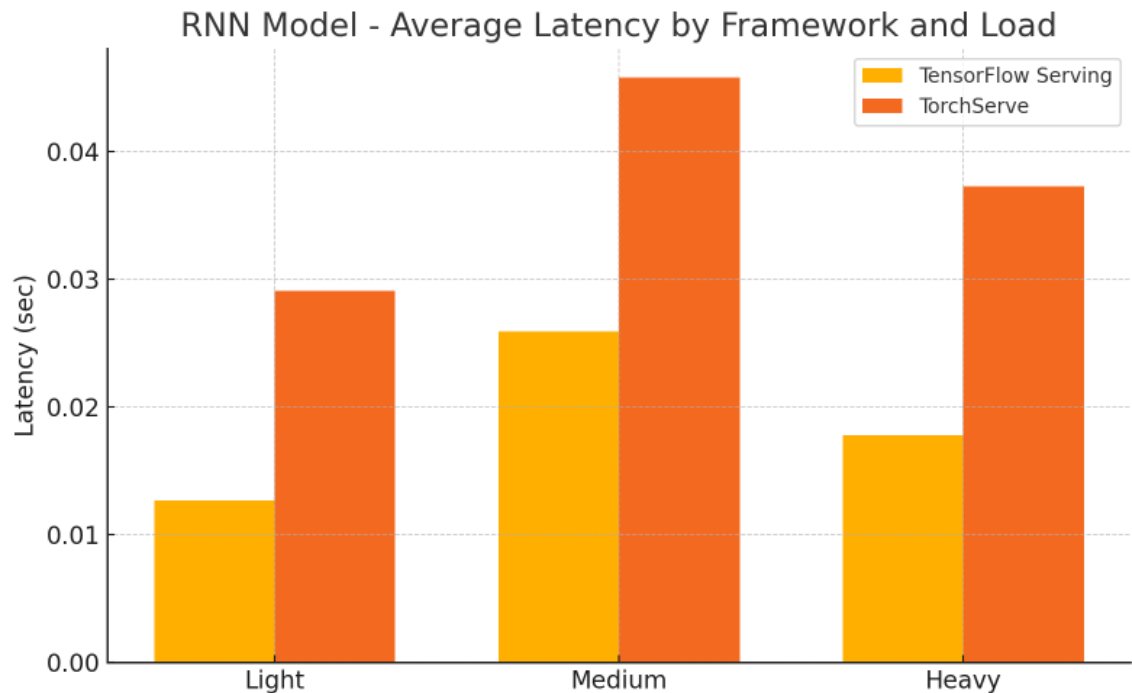


CNN Model – Throughput Comparison

- ▶ Throughput (req/s):
 - ▶ Light Load: Both 5.0
 - ▶ Medium Load: Both ~19.33
 - ▶ Heavy Load: Both 45.0
- ▶ Interpretation:
 - ▶ Both frameworks scaled well and maintained high throughput
 - ▶ Minor shortfall from 50 RPS likely due to test harness, not framework limits

RNN Model – Latency Comparison

- ▶ **Model: IMDB Sentiment Classifier (LSTM-based)**
- ▶ **TensorFlow Serving achieved significantly lower latency:**
- ▶ **Light Load: 12.7 ms (TF) vs 29.1 ms (Torch)**
- ▶ **Medium Load: 25.9 ms (TF) vs 45.8 ms (Torch)**
- ▶ **Heavy Load: 17.8 ms (TF) vs 37.3 ms (Torch)**
- ▶ **Interpretation:**
- ▶ **TF runtime better optimized for sequence models**
- ▶ **Possibly leveraging efficient CPU ops, multi-threading**



RNN Model – Throughput Comparison

- ▶ Throughput (req/s):
 - ▶ Light Load: Both 5.0
 - ▶ Medium Load: TF: 19.67 | Torch: 19.33
 - ▶ Heavy Load: Both 45.0
- ▶ Interpretation:
 - ▶ Small edge for TensorFlow in medium load
 - ▶ Faster response time enabled serving slightly more requests

Analysis & Key Takeaways

TorchServe for Vision Models

 Lower latency for CNN

 Easier debugging with TorchScript

 Customizable Python handlers

TensorFlow Serving for NLP Models

 Significantly lower latency for RNN

 Strong integration with Azure ML environments

 Optimized for sequential tasks

 Both frameworks:

 Scaled well under all load conditions

 Maintained 100% success rate

Conclusion & Recommendations



Use TorchServe for image/CNN models developed in PyTorch



Less inference latency



Better Python integration and flexibility



Use TensorFlow Serving for RNN/NLP models or TensorFlow-trained models



Lower latency and better throughput per core



Easier Azure integration



Team and ecosystem considerations matter:



Stick to native framework for smoother deployment



Future Scope:



Try higher loads, batch inference, model optimization (e.g., XLA, TensorRT)

Challenges & Solutions

▶ TensorFlow

- ▶ Model.predict() error → used model.signatures["serving_default"]
- ▶ Input shape mismatches → fixed with correct reshaping/tokenization
- ▶ Azure disk quota issues → switched to lighter base image

▶ TorchServe

- ▶ Tensor dtype mismatches → handled casting in handler script
- ▶ Manual dependency management

▶ Common

- ▶ Traffic routing issues → manually set to 100% in Azure
- ▶ Model path/version mismatches → used absolute paths

Final Summary

- ▶ Both frameworks are production-ready with proper configuration
- ▶ Choose serving stack based on:
 - ▶ Model type (CNN vs RNN)
 - ▶ Development framework (PyTorch vs TensorFlow)
 - ▶ Deployment platform (e.g., Azure ML)
- ▶ Performance depends on both model characteristics and environment setup
- ▶ Flexible integration allows for hybrid pipelines using both frameworks

Future Scope



Future Scope

- **Initial HPC Testing:**

Conducted early experiments on UH's Sabine HPC Cluster, results aligned with cloud testing. Limitations in auto-scaling and large-scale simulation led us to cloud platforms.

- **AWS Auto Scaling Goals:**

Aim to evaluate dynamic scalability using **Auto Scaling Groups (ASGs)** on AWS. This will help assess how serving frameworks handle **load-based instance provisioning**.

- **Resource Constraints:**

- Used limited **\$100 Azure student credits**, splitting deployments across accounts.

- **AWS Free Tier** did not support advanced scaling configurations.

- **Next Steps (Pre-Final Submission):**

- Evaluate **larger and ensemble ML models** for inference performance.

- Explore cloud options (e.g., **spot instances**, temporary credits) to enable **ASG-based load testing**.

- **Long-term Value:**

Insights into cost-performance tradeoffs, scalability trends, and deployment planning for **real-world traffic scenarios**.

Contributions

Team Member Responsibilities

Sai Vishnu Vamsi Senagasetty : Model training (TensorFlow & PyTorch), deployment setup on Azure ML.

Nitish dadapuram : Performance benchmarking, request simulation & data analysis, deployment

Kausthubh Reddy Eleti : Request Simulation, Data Analysis, Writing report, preparing presentation, documentation.



Thank you